Proportional Resource Scheduling on Heterogeneous Processors

Zeeshan Zaidi [1]

Abstract

Proportional-share CPU schedulers must allocate resources fairly across competing clients, but the two dominant approaches, lottery and stride scheduling, were designed for homogeneous hardware where every scheduled quantum delivers equal work. Our project implements and evaluates both algorithms across ten scenarios, measuring fairness, jitter, and convergence under static and dynamic workloads. We then extend each scheduler to heterogeneous multicore architectures, where performance cores deliver substantially more work per quantum than efficiency cores. Our key contribution is a compensated stride scheduler that weights each pass update by the assigned core's performance scale to equalize work delivered rather than quanta counted. The compensated scheduler achieves near-zero work-unit error in all static scenarios and adapts correctly to mid-simulation core throttling. We show that uncompensated stride maintains temporal fairness but accumulates systematic work-unit drift, while lottery scheduling occupies a middle ground: it cannot match compensated stride on fairness but proves more robust under high-frequency client churn and unpredictable hardware changes.

1. Introduction

A resource scheduler's effectiveness is dependent on a number of factors (Sliwko & Getov, 2025). By construction, schedulers cannot afford significant overhead, but further design concerns vary by use case. An algorithm designed for an operating system may solely prioritize its focus on responsiveness and low overhead, but schedulers for computing clusters and large data instances may also need to prioritize scalability and fault tolerance.

In any case, a fair resource scheduler must be able to allocate compute to all processes, where allocations are proportional to their needs. No process should be starved of resources, nor should any receive more than their fair share of use. The goal of this experiment is two-fold. We first want to determine what scheduling paradigms can achieve such proportional fairness. Then, we want to evaluate these schedulers in modern systems and determine how they perform in strenuous situations and under more specific use-cases.

1.1. Omitted Approaches

The archetypical schedulers of the past were multilevel feedback queues (MLFQ) (Corbató et al., 1962), Round Robin (Corbató et al., 1962), shortest job first (Cruz et al., 2020), earliest deadline first (Liu & Layland, 1973), first-in first-out (FIFO), and decay usage/priority scheduling (Epema, 1995). Each has its own drawbacks relative to our experiment.

Multilevel feedback queues incorporate multiple queues with different priority levels to handle dynamic task priorities. However, they don't allow for fixed priorities and behavior-based changes, so they're not optimal for fairness/assurances. Round Robin achieved completely fair allocation of quanta but it did not allow for varying or proportional allocation. Shortest job first carries the potential for starvation for longer jobs if many short jobs keep streaming in. Earliest deadlines first and FIFO also carry the potential to starve certain jobs. Finally, decay usage means that more usage leads to priority decay, but this experiment seeks to investigate schedulers with fixed priorities.

There are modern schedulers which we also omit from this experiment, including the Linux Completely Fair Scheduler (Pabla, 2009), Group Ratio Round Robin (GR3) (Caprita et al., 2005), and Surplus Fair Scheduling (Chandra et al., 2000). The Linux Completely Fair Scheduler introduced the concept of virtual time and used a red-black tree to reduce overhead. However, this added additional complexity and also suffered from the same issue as traditional Round Robin: a completely fair scheduler is diametrically opposed to the concept of proportional scheduling. GR3 was not considered because it deals with groups of processes, not individual processes. Finally, Surplus Fair Scheduling was also removed from potential analysis since it's geared towards multiprocessing, whereas we want to more purely analyze proportionality.

1.2. Lottery and Stride Scheduling

Lottery scheduling, introduced by Waldspurger and Weihl, represents a truly proportional-share approach (Waldspurger

& Weihl, 1994). In lottery scheduling, each runnable client is assigned a number of tickets, and the scheduler randomly selects a winning ticket each time a quantum is allocated. Intuitively, a client with more tickets should receive a larger fraction of CPU time. The main advantage with this approach is its conceptual simplicity and flexibility. Tickets can be transferred, grouped, or inflated to represent different resource policies. Its main limitation is that fairness is probabilistic, not deterministic. Thus, random variation can in fact produce bursty service, temporary inversions, and higher latency jitter even when long-run proportionality is preserved.

Stride scheduling was proposed as the deterministic alternative to lottery scheduling. Instead of randomly selecting a winner, stride scheduler assigns each client a stride value inversely proportional to its ticket count and always selects the runnable client with the smallest pass value (Waldspurger & Weihl, 1995). After every allocation, the client's pass is then incremented by its stride. In theory, this should give much tighter short-term fairness and lower variance than lottery scheduling while still preserving the same proportional-share objective.

These algorithms illustrate a fundamental tradeoff in scheduler design: randomness can provide flexibility and robustness, while determinism can improve precision and predictability. The original stride and lottery papers focus on stable proportional-share assumptions; however, real systems also face dynamic conditions such as thread creation, termination, and changing ticket assignments.

For this reason, we aim to do a comparative analysis of the two schedulers under both static and dynamic workloads to assess whether the classical theoretical claims hold, where each approach performs better, and where limitations become visible.

1.3. Gap in Original Papers

Though we can investigate both schedulers by simply implementing their papers directly, the original works are predicated on the assumption that resource shares are equal in any given system. Particularly, in a chip all cores are equal. This assumption was true during the development of stride and lottery schedulers, but modern chips are now equipped with performance (P) cores who can execute far more in a single quantum than an efficiency (E) core. As a result, scheduling algorithms which allocate resource share based on time may not be "fair" on modern chips.

Thus to appropriately compare the scheduling approaches, we must also adapt them to heterogeneous processing units. To do so, we attempt to define an exchange rate between cores that is proportional to their computational power, and incorporate that into the scheduling algorithms.

Thus, our experiment adheres to the following workflow:

1. Implement vanilla stride and lottery scheduling algorithms
2. Evaluate schedulers on homogeneous processing units
3. Modify schedulers for systems with varied processing power
4. Evaluate schedulers on heterogeneous processing units

2. Related Work

2.1. Foundational Proportional-Share Scheduling

The two schedulers studied in this work were introduced by Waldspurger and Weihl in a pair of companion papers. The lottery scheduling paper (Waldspurger & Weihl, 1994) establishes that wins follow a binomial distribution, giving expected error $O(\sqrt{n})$ over n quanta. The stride scheduling paper (Waldspurger & Weihl, 1995) provides a stronger guarantee: absolute allocation error is bounded at ≤ 1 quantum per client (Theorem 1), and the hierarchical variant tightens this to $O(\log n_c)$. Nevertheless, both papers assume homogeneous hardware whereby every scheduled quantum delivers equal work, which is not true in modern systems.

2.2. Heterogeneous Processor Architecture

Modern hybrid processors, typified by Intel's Alder Lake architecture, pair high-performance cores (P-cores) with energy-efficient cores (E-cores) that deliver substantially less computational work per clock cycle. Intel's Hardware Feedback Interface (HFI) (Rukmabhatla et al., 2021) exposes per-core performance and efficiency capability scores updated in hardware every $\sim$4ms. Measured values for Alder Lake place P-cores at approximately 255 and E-cores at 128 capability units on compute-intensive workloads, a roughly 2:1 ratio. This directly motivates our choice of $S_P = 2.0$ and $S_E = 1.0$ as the performance scale parameters in our heterogeneous extension.

Van Craeynest et al. (Van Craeynest et al., 2012) introduced CPI-stack decomposition to predict per-thread performance on each core type without requiring direct measurement on both, a precursor to Intel Thread Director. In a production deployment, per-core performance scales (denoted S_C for core C) would be estimated per-thread by mechanisms of this kind, however our simulator simply treats S_C as a configured parameter based on the HFI capability ratios above.

2.3. Heterogeneous Scheduling Policies

The systems literature has responded to hardware heterogeneity primarily through *assignment policy*: deciding

which thread should run on which core type. Koufaty et al. (Koufaty et al., 2010) use hardware performance counters to classify threads as compute-bound or memory-bound and bias their assignment to the appropriate core type, improving throughput on Intel Sandy Bridge. This work is orthogonal to ours: Koufaty et al. optimize the placement decision, whereas we take placement as given (random) and correct for heterogeneity in the scheduler's accounting.

At the operating system level, Linux Energy Aware Scheduling (EAS) (Linux Kernel Developers, 2023) assigns tasks to cores using a platform energy model and per-core capacity values directly analogous to our S_C. EAS operates at the runqueue and task migration level, assuming some underlying proportional-share mechanism handles allocation among tasks co-scheduled on a given core. Our work targets that underlying mechanism: we extend the formal guarantees of Waldspurger's model to the heterogeneous setting that EAS presupposes.

2.4. Proportional-Share Scheduling on Heterogeneous Hardware

The closest prior work to our extension is Lucky Scheduling (Petrucci et al., 2012), which applies lottery scheduling to heterogeneous multi-core processors using MIPS-based ticket scaling. Lucky Scheduling creates a separate lottery pool per core and scales each pool's effective ticket count by the core's MIPS rating. It achieves per-quantum spatial fairness through structured assignment.

Our approach takes a different philosophy: rather than partitioning clients across per-core pools, we maintain a single global pool and compensate for heterogeneity in the scheduler's internal accounting after each allocation. This keeps the client model intact and allows the same compensation mechanism to be applied uniformly to both lottery and stride scheduling, whereas Lucky Scheduling addresses lottery only.

For stride scheduling, this single-pool compensation integrates naturally with the existing pass/remain mechanism. Preserving this invariant under heterogeneous hardware, including mid-simulation changes to core performance, is a central contribution of our work. For lottery scheduling, the stateless nature of the algorithm means compensation cannot be applied retroactively. Instead, we show empirically how the independent draw model fares against stride under the same heterogeneous conditions.

This design also yields a formal guarantee absent from Lucky Scheduling: with maximum core scale $S_{\max}$, work-unit allocation error is bounded at $\leq S_{\max}$ times Waldspurger's original per-client error bound, tightening automatically as cores throttle.

3. Design

3.1. Base Implementations

Both the original lottery and stride schedulers are implemented, in all forms, by extending our abstract Scheduler class. A Scheduler uses a SimClient struct, which includes various details for a client: client id, client name, and assigned ticket count. We use SimClients in place of actual client processes for the sake of simplicity, as this representation is sufficient in testing scheduler behavior.

The two forms of the lottery scheduler (Waldspurger & Weihl, 1994) are very simple in their resource allocation mechanism: they generate a random ticket, identify the client that holds that ticket, and allocate resources for that client. They differ in how they identify the winner of their lotteries. The standard list-based lottery scheduler (`lottery_scheduler_list`) simply iterates through an ordered list of all clients, adding each client's number of tickets to a running sum until the correct client is found. In contrast, the tree-based scheduler (`lottery_scheduler_tree`) simulates a binary tree by storing ticket counts in a 1-indexed flat array, where the root is at index 1. We define `capacity_` to be the number of leaf nodes needed to store all clients, rounded up to the nearest power of 2, and begin storing clients at index capacity. Internal nodes store the sum of their subtrees. This tree is traversed to achieve $O(\log n)$ runtime, a valuable optimization compared to the $O(n)$ search time needed for the standard lottery scheduler.

Our base stride scheduler (`stride_scheduler`) (Waldspurger & Weihl, 1995) adds more information to their clients: stride, calculated using $\texttt{STRIDE1} = (1 \ll 20)/\texttt{tickets}$, and a pass counter that advances by stride every time the client is chosen. Because of the difference in algorithm, the winning client is simply the one with the smallest pass. We also implemented dynamic client participation by allowing clients to leave and join the scheduler as they please; these functions preserve the client's position in its stride cycle when it leaves and ensures that it is not unfairly allocated resources repeatedly to "catch it up" when it re-joins. A `global_pass` is also tracked upon every join and leave to ensure that clients are re-inserted at a fair position.

However, the base stride scheduler algorithm does not mitigate absolute throughput error issues that can become an issue when the number of clients grows too large. One example that Waldspurger and Weihl cite is a scenario in which 101 clients are given a 100:1:...:1 ticket allocation (Waldspurger & Weihl, 1995). In such a scenario, the intended proportion of resources for the 100-ticket client is 50, but in the standard stride scheduler, it would receive 100 allotments before any other client receives one – not

the intended outcome. Our hierarchical stride scheduler (`stride_scheduler_hierarchical`) (Waldspurger & Weihl, 1995) aims to resolve this issue by splitting clients into a balanced binary tree. This tree's leaf nodes are individual clients, while its internal nodes aggregate the tickets of their children and track the minimum of their passes. An allocation call descends from the root and always chooses the child with the smaller pass until a leaf node is reached, then advances that leaf's pass by its stride and propagates new pass values upwards. Clients are implicitly grouped by their position in the tree, so higher-ticket clients that should dominate allocation compete at each level of the tree; this prevents issues of unintentionally low allocation experienced using a standard scheduler. It is worth noting that our implementation requires a full reconstruction of the balanced tree upon every addition, removal, modification, join, or leave; being an O(n) operation, this could be a severe flaw in performance for a real system with many clients.

3.2. Modified Schedulers

To evaluate scheduling on heterogeneous hardware, we extend our schedulers to operate across a pool of cores with differing performance characteristics. Each core C is assigned a performance scale $S_C \in \mathbb{R}_{>0}$, where $S_C = 2.0$ for a P-core and $S_C = 1.0$ for an E-core, reflecting the roughly 2:1 single-threaded performance advantage observed in Intel's 12th Generation hybrid architecture (Rukmabhatla et al., 2021). The simulator models four P-cores and four E-cores, matching the Alder Lake consumer configuration. S_C is mutable at runtime, enabling mid-simulation events such as thermal throttling.

A multi-core scheduler must produce N allocations per quantum (one per core) rather than the single allocation of the base schedulers. We extend each scheduler with an `allocate_multi` method that returns a list of (client, core) pairs. Core assignment is randomized each quantum via a shuffle, so no client is systematically favored toward P-cores or E-cores.

3.2.1. Compensated Stride

Though the hierarchical scheduler achieves a tighter theoretical error bound, we do not carry it forward into the heterogeneous extension. Its O(n) tree rebuild on every client join, leave, or modification proved brittle under dynamic workloads in our thread-thrashing scenario (see Evaluation). Extending the hierarchical structure to heterogeneous hardware would further require rescaling aggregate pass values at internal nodes on every core-scale change, compounding this fragility. Meanwhile, the base dynamic stride scheduler provides a cleaner invariant structure and handles all dynamic scenarios with lower overhead. Essentially, the marginal error-bound improvement of the hierarchical variant does not justify these costs.

The compensated stride scheduler (`hetero_stride`) modifies the pass update rule to account for the work value of the assigned core. After a client wins a core C, its pass advances by:

$$\textit{pass} \mathrel{+}= \textit{stride} \times S_C \tag{1}$$

In practice this is implemented in integer fixed-point arithmetic ($\textit{pass} \mathrel{+}= (\textit{stride} \times \lfloor S_C \times 1000 \rfloor)/1000$) to avoid floating-point mixing with the pass accumulator. Intuitively, winning a P-core advances a client's virtual clock twice as far as winning an E-core. This creates a negative feedback loop: a client that receives P-core time becomes deprioritized for subsequent allocations, allowing other clients to receive more core time until work-unit proportions are restored.

The global pass similarly advances in work units rather than time units, at a rate proportional to total work dispensed per quantum. Client join and leave semantics are inherited unchanged from the base stride scheduler; the remain invariant (pass $-$ global_pass $=$ credit) is preserved across both events.

A new invariant arises under dynamic S_C changes. When a P-core throttles mid-simulation, the global work rate decreases and inactive clients' remain values which were computed under the old work rate become miscalibrated. We correct this via `rescale_remains(ratio)`, which multiplies each inactive client's remain by the ratio of new to old total work capacity, restoring the invariant without disrupting active clients' pass values.

The formal error bound extends naturally from the original stride guarantee. Since each pass increment is scaled by at most $S_{\max}$, the maximum work-unit divergence from the intended allocation per client per round is bounded by $S_{\max} \times 1$ work unit. For our configuration with $S_{\max} = 2.0$, the bound is 2 work units per client per round, tightening automatically when P-cores throttle.

3.2.2. Uncompensated Stride (Baseline)

The uncompensated stride scheduler (`homo_stride`) is structurally identical to the compensated variant but advances pass by stride only, ignoring S_C:

$$\textit{pass} \mathrel{+}= \textit{stride} \tag{2}$$

Global pass tracks time units rather than work units. This scheduler achieves near-perfect temporal fairness, meaning clients receive allocations proportional to their ticket counts, but accumulates systematic work-unit error because it cannot distinguish between winning a P-core and an E-core. It serves as a controlled baseline that isolates the contribution of S_C compensation to work-unit fairness.

3.2.3. HETEROGENEOUS LOTTERY

Both lottery implementations are statistically identical when seeded identically, as verified empirically in our equivalence scenario. We carry only the tree-based scheduler forward into the heterogeneous extension. With N cores, the heterogeneous scheduler performs N draws per quantum. This means the O($\log n$) draw cost of the tree compounds more favorably than the O(n) linear scan of the list implementation, and the segment tree structure is already required to support efficient dynamic ticket updates. The list-based scheduler is retained in the base evaluation to confirm this equivalence and serves no further purpose beyond it.

The heterogeneous lottery scheduler (`hetero_lottery`) performs N independent draws with replacement per quantum, one per core. Each draw selects a winner proportionally to ticket count from the same segment tree used in the base tree lottery. The same client may win multiple cores in a single quantum. Core assignment is randomized, so in expectation each client receives P-core and E-core time proportional to its ticket share. However, because lottery is stateless, there is no mechanism to correct for imbalances that arise from random core assignment variance, and no analog to the remain invariant that could be rescaled on a throttle event.

3.3. Event Simulator

Events are simulated in `event_simulator`, which processes JOIN, LEAVE, and MODIFY events, calls appropriate scheduler functions at the proper quanta, and records `ClientTracks` for each client. `ClientTracks` store win counts, first and last active quanta, and lists of inter-win intervals. The test scenarios are run in `main.cpp`. These tests construct schedulers, define `SimClients` and sequences of events to be tested, then pass these to the simulator, which returns a `SimResult` containing the history and client tracking information. These results are then used to compute all of the metrics discussed in the next section.

4. Methodology

Our experiments use independent test scenarios to compare the behavior of the scheduler implementations described above. We first analyze the results of each individual test before drawing larger conclusions about the behavior of our schedulers.

Each test scenario measures five key metrics of the schedulers' behavior and their resource allocation relative to ground-truth fairness. Throughout, i denotes the client index, t_i its ticket count, T the total tickets across all active clients, n_a the number of quanta elapsed while client i is active, w_i the quanta won by client i, and Δ the convergence window width.

Metric	Formula
Absolute Error	$\lvert w_i - n_a \cdot t_i/T \rvert$
Pairwise Relative Error	$\max_{(i,j)} \left\lvert w_i - (w_i + w_j) \cdot \frac{t_i}{t_i+t_j} \right\rvert$
Throughput Ratio	$w_i \ / \ n_a$
Jitter	σ(inter-allocation intervals)
Convergence Window	$\frac{1}{W} \sum_{W} \lvert W_i - (t_i/T) \cdot \Delta \rvert$

Table 1. Base evaluation metrics.

Absolute error measures how far client i's actual win count deviates from its ticket-proportional entitlement over the quanta it was active. *Pairwise relative error* captures the worst-case fairness violation across any pair of clients, isolating relative bias independently of total quanta. *Throughput ratio* is the fraction of active quanta awarded to client i; the ideal value is t_i/T. *Jitter* is the standard deviation of inter-allocation intervals where a value of zero means wins are perfectly periodic. *Convergence window* reports the mean absolute error per fixed window of width Δ, averaged across all windows W. Smaller windows indicate faster convergence to steady-state proportional allocation.

4.1. Heterogeneous Metrics

For heterogeneous scenarios, three additional metrics track work-unit fairness across cores with differing performance scales. Let S_C denote the performance scale of core C, and let $\mathcal{W}_i = \sum_{\text{wins of } i} S_C$ be the total effective work received by client i. Let $\mathcal{W} = \sum_i \mathcal{W}_i$ be the total work dispensed.

Metric	Formula
Effective Work	$\mathcal{W}_i = \sum_{\text{wins}} S_C$
Computational Error	$\lvert \mathcal{W}_i - (t_i/T) \cdot \mathcal{W} \rvert$
Work Throughput Ratio	$\mathcal{W}_i \ / \ \mathcal{W}$

Table 2. Heterogeneous evaluation metrics.

Effective work sums the performance scale of every core won by client i, converting quantum counts into actual compute delivered. *Computational error* measures deviation from the client's intended work share $(t_i/T) \cdot \mathcal{W}$, the heterogeneous analog of absolute error. *Work throughput ratio* is the fraction of total dispensed work received by client i, where the ideal value is t_i/T, matching the ticket proportion. All five base metrics are also reported for heterogeneous scenarios to enable direct comparison with the homogeneous results.

5. Evaluation

5.1. Homogeneous Scenarios

5.1.1. Scenario 1 — Static 7:3, 1000 quanta

The first test aims to establish an accuracy baseline for the schedulers by replicating Figure 9 of the stride paper (Waldspurger & Weihl, 1995). It mocks 2 clients with a ticket ratio of A:B = 7:3. All schedulers achieved near-exact proportionality (A: 700–702 wins, B: 298–300 wins), but with notable gaps in quality of service. The lottery scheduler produced jitter values of 0.74 and 2.75 for clients A and B respectively, while both stride schedulers held at about 0.50 for each. This indicates that, though lottery achieved fair allocation, the consistency of its service to clients varies widely compared to stride. This could be problematic for real applications which require regular resource use. This phenomenon is made clearer by the convergence behavior of the schedulers, where lottery scheduling fails to maintain steady state exactly.

Scheduler	A Jitter	B Jitter	PairErr	Conv. 50q
Lottery	0.74	2.75	2.00	0–7, oscillates
Stride	0.50	0.47	1.00	0 from window 2
Stride (H)	0.50	0.47	0.00	0 throughout

Table 3. Scenario 1 results.

5.1.2. Scenario 2 — Static 19:1, 1000 quanta

To further measure consistency of service, the second test mocks two clients at a severely skewed ticket ratio of 19:1. This means that client B should only win a resource share once every twenty quanta on average, about 50 total wins. Again, results indicate that while all schedulers reach this near-fair proportion, lottery scheduling provides inconsistent gaps in service to the lower priority client. Both stride variants achieved a mean response time of exactly 20.00 quanta for client B, matching the theoretical 1/20 period; lottery's mean of 21.63 reflects the geometric tail of its inter-win distribution. On the contrary, both stride implementations are able to provide periodic service to the same client.

Scheduler	B wins	B Jitter	PairErr
Lottery	47	19.29	3.00
Stride	49	0.00	1.00
Stride (H)	50	0.00	0.00

Table 4. Scenario 2 results (B = minority client, 1 ticket).

5.1.3. Scenario 3 — Static 3:2:1, 100 quanta

This test replicates a short window, using 100 quanta rather than 1000, with 3 clients whose ticket ratio is 3:2:1. It demonstrates that short windows are more damaging to lottery schedulers than stride, as there is not enough time for the variation of lottery assignments to converge to fair allocations. As expected, stride does not exhibit this weakness due to its deterministic nature. The results below demonstrate lottery's inability to fully reach fair allocations in the limited timeframe. Most strikingly, the lottery's B-client (2 tickets) outright exceeded the A-client (3 tickets) in wins — a direct ordering inversion that stride never exhibits.

Scheduler	A wins	B wins	C wins	PairErr
Lottery	41	42	17	8.80
Stride	51	33	16	0.75
Stride (H)	51	33	16	0.75
Intended	50	33.3	16.7	—

Table 5. Scenario 3 results.

5.1.4. Scenario 4 — Dynamic ticket modification, 1000 quanta

This test mocks mid-run ticket changes, with Client A's ticket allocation being redrawn from 2–12 every 2 quanta while client B holds fixed at 3. Lottery awarded client A 675 wins and B 325; stride reversed this to 336 and 664. Lottery favors A because its draw is weighted by the instantaneous ticket count each quantum, and A spends more of the cycle holding high tickets (up to 12) than B's fixed 3, so the average draw tilts toward A. The stride reversal occurs because stride's rescaling mechanism responds to A's instantaneous ticket count at each modification: during low-ticket phases, A's stride grows large and its pass advances rapidly, accumulating a deficit that high-ticket phases cannot fully recover. Since absolute error for client A is constantly changing with its ticket allocation, the meaningful observation in this test case emerges in the convergence window behavior. Lottery scheduling hovered from 9–20 quanta of error per 50-quanta window, while stride was much tighter ranging from 3–4. It demonstrates that stride's rescaling mechanism handles perturbation with less error than lottery's stateless redraw.

Scheduler	B AbsErr	Conv. Window Range
Lottery	275	9–20 per 50q window
Stride	64	3–4 per 50q window

Table 6. Scenario 4 results.

5.1.5. Scenario 5 — Skewed 8-client H(7):L1–L7(1), 10000 quanta

This scenario directly reproduces a comparison of dynamic and hierarchical stride algorithms from the stride paper (Figure 13, Section 5.4) (Waldspurger & Weihl, 1995). Lottery is excluded because this scenario's sole purpose is to validate the stride paper's theoretical error bounds. Specifically,

the test aims to show the error bound of each algorithm and demonstrate that hierarchical stride achieves a lower max pairwise relative error than traditional stride. Both schedulers achieve identical mean response times (H: 2.00 quanta, L: 14.00 quanta) and proportional accuracy. The distinguishing factor is pairwise error, as hierarchical stride's error is three times lower, consistent with the paper's theoretical bounds of $O(\log n_c)$ vs. $O(n_c)$.

Scheduler	H wins	PairErr
Stride	5003	1.50
Stride (H)	5002	0.50

Table 7. Scenario 5 results (H = heavy client, L = light clients).

5.1.6. Scenario 6 — Dynamic join/leave, 500 quanta

This scenario measures the effect of clients joining and leaving during execution. Specifically, two permanent clients, A (6 tickets) and B (4 tickets), begin alone before clients C (5 tickets) and D (2 tickets) join during execution. The results demonstrate that lottery scheduling handles these changes very smoothly due to its stateless pool adjustment. On the contrary, stride's mechanism cannot eliminate the distortion to client allocations in the timeframe, though it does correctly prevent the new clients from monopolizing resources with a low stride value. Stride's worse performance is demonstrated by the higher error values for clients A and B, along with its convergence window measurements.

5.1.7. Scenario 7 — Lottery List vs. Tree equivalence + timing, 10000 quanta

This test case verifies our two lottery scheduler implementations, linear and tree, to ensure they are statistically equivalent. Using the same random seed, both implementations produced identical win counts (A: 4967, B: 3005, C: 2028), confirming statistical equivalence. Simultaneously, a clock timer verifies that the tree-based implementation is faster, even at this scale.

5.1.8. Scenario 8 — Thread thrashing, 600 quanta

This test mocks high-frequency client entrance and exit. Two stable clients, A and B, each have 4 tickets and remain in the process over the entire time period. Meanwhile, three churning clients C1, C2, and C3 cycle in and out of the process every 20 quanta in staggered phases. As seen in Scenario 6, lottery scheduling proves far more structurally robust, allowing clients A and B to achieve almost equal resource shares. On the other hand, stride performs extremely poorly. Though A and B split wins nearly equally between themselves (78 and 77 respectively), both totals fall far below their proportional entitlement as the churning clients crowd them out. Stride also exhibits higher jitter and a worsening convergence window over time.

5.1.9. Scenario 9 — Multiprocess ticket inflation, 1000 quanta

This test uses the ticket inflation problem from the lottery paper to test the relative robustness of the stride and lottery algorithms (Waldspurger & Weihl, 1994). Essentially, there are 2 processes, P1 and P2, each with 6 tickets. P1 has three threads, each with 2 tickets of its 6, while P2 is single-threaded. In Phase A (quanta 1–500), both schedulers correctly delivered near-equal allocation (Lottery: 49.2%/50.8%, Stride: 50.2%/49.8%). In Phase B, beginning at quantum 501, P1 spawns another thread with 2 tickets, taking P1's total allocation to 8. P2 remains the same, so the desired allocation is 57.1% to 42.9%. Lottery nearly achieves this, but stride fails to adjust. The new thread joins and aggressively occupies resource share, causing P1 to reach 64% of resource share. Again, results demonstrate stride scheduling failing to adjust to dynamic joins.

5.1.10. Scenario 10 — Lottery convergence rate ($1/\sqrt{n}$ law), 200 trials × 5 n-values

The lottery paper asserts that wins should follow a binomial distribution, implying that lottery's convergence rate should be $1/\sqrt{n}$, where n is the number of quanta (Waldspurger & Weihl, 1994). Since a single run is not enough to confirm this phenomenon, the final test runs a 7:3 client ratio at five log-spaced values of n (100, 316, 1000, 3162, 10000) with 200 independent RNG seeds per n. The mean absolute error across all five n values was 0.375 and within 2.5% of the theoretical prediction, which we found sufficient to confirm the scaling law.

5.2. Heterogeneous Scenarios

Three schedulers are compared throughout: compensated stride (Stride C), which advances pass by *stride* $\times S_C$; uncompensated stride (Stride U), which advances pass by stride only and serves as a temporal-fairness baseline; and tree-based lottery (Lottery). All scenarios run on 4 P-cores ($S_C = 2.0$) and 4 E-cores ($S_C = 1.0$), yielding a total work rate of 12 work units per quantum pre-throttle.

5.2.1. H1 — Proportional baseline: A(7):B(3), 1000 quanta

This test replicates the static 7:3 scenario on heterogeneous hardware, establishing a baseline for the work-unit metrics. All three schedulers achieve near-correct temporal win proportions, but diverge sharply on work-unit fairness. Compensated stride delivers exactly 70% and 30% of total work

Scheduler	A AbsErr	A Jitter	B AbsErr	B Jitter	Peak Conv.
Lottery	48	1.32	31	2.62	11.5
Stride	107	7.67	73	12.33	25.0

Table 8. Scenario 6 results.

Scheduler	PairErr	Max Jitter	Clock time
Lot. (List)	29.43	4.46	0.595 ms
Lot. (Tree)	29.43	4.46	0.540 ms

Table 9. Scenario 7 results.

to A and B (WorkTpR = 0.700/0.300), achieving CompErr = 0.00. This comes at a small temporal cost: AbsErr = 20.00, because some of A's wins land on P-cores delivering more work per win, so the scheduler steers subsequent wins to B, intentionally reducing A's win count below the temporal ideal. Uncompensated stride achieves perfect temporal fairness (AbsErr = 0.00, wins 5600:2400) but accumulates CompErr = 33.00, showing the algorithm cannot detect the imbalance of random core assignment. Lottery falls between the two (CompErr = 23.00) with slightly higher B jitter (0.53 vs. 0.49).

5.2.2. H2 — MULTI-CLIENT {5,4,3,2,1}, 5000 QUANTA

H2 confirms that compensated stride generalizes to 5 clients with non-uniform ticket counts. Compensated stride achieves CompErr = 0.00 for all five clients, delivering exactly the work-unit share proportional to each client's tickets. The temporal AbsErr values (14.67–74.33) are the expected consequence of the compensated scheduler adjusting win counts to equalize work, not temporal fairness. Uncompensated stride maintains near-perfect temporal fairness (max AbsErr = 0.67) but shows persistent work-unit drift: CompErr ranges from 2.00 to 33.00, with higher-ticket clients accumulating more absolute drift because they receive more total allocations.

Lottery performs worst on both metrics, with max CompErr = 241.00 for the highest-ticket client. Jitter also increases sharply for minority clients: client E (1 ticket) shows jitter = 1.85 versus A's 0.52, confirming that lottery's randomness disproportionately affects low-priority clients. Lottery's work-unit error is not a convergence phenomenon that more quanta would resolve. This is distinct from the temporal $1/\sqrt{n}$ convergence confirmed in Scenario 10: win *counts* do converge, but the *work value per win* varies with the randomly assigned core, and lottery has no mechanism to correct this per-win imbalance. Instead, it shows lottery is akin to a random walk without a restoring force. Each quantum's random core assignment adds independent noise to each client's work received, and without a correction mechanism this drift accumulates proportionally to $\sqrt{\text{quanta}}$.

5.2.3. H3 — DYNAMIC JOIN/LEAVE: A(6) B(4) + C(5) Q200–400, D(2) Q300+

H3 tests whether the remain mechanism and work-unit pass accounting function correctly under dynamic pool changes. The headline AbsErr and CompErr values for A and B are large across all schedulers (AbsErr ∼290–440, CompErr ∼410–690), but this is a metric artifact mirroring the behavior seen in homogeneous Scenario 6: intended work is computed from the initial pool ($T = 10$), but during q200–400 the pool expands to include C and D, diluting each client's fair share. The error reflects ticket pool volatility, not scheduling failure.

The convergence windows reveal the true behavior by phase. In the static opening (q1–200), compensated stride shows CompErr = 0.0–2.5 per window while uncompensated shows 2.5–10.0 and lottery 2.5–13.0. During the transition phase (q200–400), all three schedulers show elevated error (93–135) as the pool changes dominate. After the pool stabilizes (q400–500), the schedulers converge similarly as settlement time dominates over compensation. The elevated CompErr values (49–51) in this final phase reflect the metric artifact from the initial ticket pool persisting across windows. The within-phase scheduling behavior is correct.

5.2.4. H4 — P-CORE THERMAL THROTTLE: A(6):B(4), $S = 2.0 \rightarrow 1.4$ AT Q300

H4 is the most important heterogeneous scenario: it validates the `rescale_remains()` mechanism under a mid-simulation `SCALE_CHANGE` event. At quantum 300, all four P-cores throttle from $S = 2.0$ to $S = 1.4$, reducing the total work rate from 12.0 to 9.6 work units per quantum. Compensated stride achieves CompErr = 0.36, a near-zero value despite the mid-run change. Its WorkTpR values (0.5999/0.4001) match the 6:4 ticket ratio almost exactly. The formal error bound also tightens automatically: with $S_{\max}$ dropping from 2.0 to 1.4, the bound reduces from 2.0 to 1.4 work units per client per round. Uncompensated stride accumulates CompErr = 33.96 despite perfect temporal fairness (AbsErr = 0.00). The drift persists post-throttle because the scheduler has no mechanism to detect work-unit imbalance. Lottery achieves CompErr = 9.16, better than uncompensated due to partial cancellation from random core assignment variance, but far from the compensated result.

Scheduler	A AbsErr	A Jitter	B AbsErr	B Jitter	Conv. Windows
Lottery	73	2.04	86	2.12	3–4, stable
Stride	222	3.24	223	2.97	2.5–8, drifting

Table 10. Scenario 8 results.

Scheduler	CompErr	WorkTpR A	WorkTpR B
Stride C	0.00	0.700	0.300
Stride U	33.00	0.697	0.303
Lottery	23.00	0.698	0.302

Table 11. H1 results.

Scheduler	Max CompErr	Conv. Window Range
Stride C	0.00	0.0–0.8
Stride U	33.00	1.6–5.6
Lottery	241.00	4.4–15.6

Table 12. H2 results (max over all clients).

Phase	Stride C	Stride U	Lottery
q1–200 (static)	0.0–2.5	2.5–10.0	2.5–13.0
q200–400 (transition)	99–123.5	97.5–124.0	93.5–135.0
q400–500 (settling)	49.5–50.0	47.0–51.5	39.5–50.5

Table 13. H3 mean CompErr per convergence window, by phase.

Scheduler	CompErr	WorkTpR A	WorkTpR B
Stride C	0.36	0.5999	0.4001
Stride U	33.96	0.5948	0.4052
Lottery	9.16	0.5986	0.4014

Table 14. H4 results.

6. Conclusion

This experiment set out to evaluate stride and lottery scheduling under both static and dynamic workloads, and to determine whether each approach could be adapted to deliver proportional fairness on heterogeneous hardware.

6.1. Stride Scheduling

6.1.1. Stride Implementations

Both stride implementations correctly enforce proportional allocation in static scenarios. The hierarchical variant achieves a tighter $O(\log n_c)$ pairwise error bound compared to dynamic stride's $O(n_c)$, confirmed in Scenario 5 where its MaxPairRelErr was three times lower. However, hierarchical stride requires a full $O(n)$ tree rebuild on every client join, leave, or modification, making it very brittle under high-churn workloads. Extending the hierarchical structure to heterogeneous hardware would further require propagating `rescale_remains()` corrections through aggregate internal nodes on every `SCALE_CHANGE` event, compounding this fragility. Dynamic stride's cleaner invariant structure made it the natural choice to carry forward, with the marginal error-bound improvement of the hierarchical variant not justifying the added complexity.

6.1.2. Stride Performance

In homogeneous scenarios, stride scheduling validates its theoretical guarantees across all static workloads. Jitter values were consistently low (0.47–0.50 across all ticket ratios), convergence windows stabilized within one to two rounds, and win counts closely matched ticket-proportional entitlements. These strengths hold as long as the client pool is stable. Under dynamic conditions, a consistent weakness emerges: in both the join/leave scenario (Scenario 6) and the thread-thrashing scenario (Scenario 8), stride's remain invariant allows rejoining clients to dominate allocation, crowding out stable clients. Lottery outperformed stride substantially in both cases. This pattern carries directly into the heterogeneous extension, where stride's deterministic invariant enables precise work-unit compensation in stable configurations but remains vulnerable to rapid pool changes.

Compensated stride's core success is the pass update rule $pass \mathrel{+}= stride \times S_C$, which creates a deterministic negative feedback loop: a client that wins a P-core accumulates pass faster, reducing its priority for subsequent allocations until work-unit proportions are restored. This mechanism achieves CompErr = 0.00 in all static heterogeneous scenarios and CompErr = 0.36 under mid-simulation thermal throttle, translating the original temporal guarantee of stride scheduling into a work-unit guarantee on heterogeneous hardware. The formal error bound extends cleanly: work-unit error is bounded by $S_{\max} \times 1$ work unit per client per round, and tightens automatically when cores throttle. Generally, compensated stride is best suited to environments where work-unit proportionality is the primary objective and the client pool is relatively stable, as it still falls short under high-frequency client churn. As demonstrated in Scenario 8, the remain invariant that enables fair re-entry under mild pool changes becomes a liability when clients cycle in and out rapidly, allowing rejoining clients to temporarily dominate allocation and starve stable clients.

6.2. Lottery Scheduling

6.2.1. Lottery Implementations

Both lottery implementations produce statistically identical results under the same random seed, confirmed in Scenario 7. They differ only in efficiency, as the list-based scheduler scans clients in $O(n)$ per draw while the tree-based scheduler traverses a segment tree in $O(\log n)$. In the heterogeneous extension, where N draws are required per quantum (one per core), this difference compounds favorably for the tree and the segment tree structure is already required to support efficient dynamic ticket updates. The list-based scheduler was retained solely to verify equivalence in Scenario 7 and serves no further purpose beyond that confirmation.

6.2.2. Lottery Performance

In homogeneous scenarios, lottery scheduling confirms both its theoretical guarantees and its known limitations. The $1/\sqrt{n}$ convergence law was validated empirically across five log-spaced quanta values, and both implementations produced identical results under the same random seed. Proportional accuracy was achieved in all static scenarios, but at the cost of higher jitter — most dramatically in the 19:1 scenario, where lottery's B-client jitter reached 19.29 versus stride's 0.00. Short simulation windows exacerbated this further: in Scenario 3 (100 quanta), lottery's pairwise error reached 8.80 versus stride's 0.75. Where lottery proved consistently stronger was in dynamic scenarios: in both Scenario 6 and Scenario 8, lottery's stateless pool adjustment absorbed client arrivals and departures cleanly, while stride's stable clients were crowded out. This structural robustness is lottery's primary advantage in the heterogeneous setting as well.

Lottery scheduling is not competitive with compensated stride on work-unit fairness in static or mildly dynamic scenarios. Its CompErr accumulates as a random walk proportional to $\sqrt{\text{quanta}}$. This is because random core assignment adds independent noise at each quantum with no restoring force, so longer simulations widen rather than narrow the work-unit error. Nevertheless, lottery retains genuine structural advantages that make it preferable in specific operating regimes. Because it is completely stateless between draws, there is no remain invariant to corrupt under rapid client churn, the condition where stride performs most poorly. Lottery also requires no `rescale_remains()` correction when S_C changes. Instead, it simply draws from whoever is in the pool with whatever tickets they hold, making it inherently robust to unpredictable core performance changes. In the event of a scheduler restart or process migration, lottery resumes correctly on the next draw with no reinitialization, whereas compensated stride loses all pass and remain state. Lottery scheduling is therefore best suited to high-churn environments and to systems where approximate work-unit fairness is acceptable in exchange for robustness and implementation simplicity.

The heterogeneous lottery results also validate the premise of Lucky Scheduling (Petrucci et al., 2012). In H4, lottery achieves CompErr = 9.16 (compared to uncompensated stride's 33.96) without any compensation mechanism because random core assignment partially cancels over many draws, distributing P-core and E-core time stochastically across clients. Lucky scheduling formalizes this observation by embracing the residual randomness as a feature rather than a flaw, trading precise work-unit fairness for energy efficiency. Our results confirm that the implicit compensation is real and non-trivial, suggesting that in systems where energy management is a primary concern alongside approximate proportionality, lottery-style scheduling on heterogeneous hardware remains a competitive and principled approach.

References

Caprita, B. et al. Group ratio round-robin: O(1) proportional share scheduling for uniprocessor and multiprocessor systems. In *USENIX Annual Technical Conference, General Track*. USENIX Association, 2005.

Chandra, A. et al. Surplus fair scheduling: A proportional-share CPU scheduling algorithm for symmetric multiprocessors. In *Fourth Symposium on Operating Systems Design and Implementation (OSDI 2000)*. USENIX Association, 2000.

Corbató, F. J., Merwin-Daggett, M., and Daley, R. C. An experimental time-sharing system. In *Proceedings of the May 1–3, 1962, Spring Joint Computer Conference*, pp. 335–344. ACM, 1962.

Cruz, K. C. et al. An experimental study on the shortest fittest job first (SFJF) scheduling algorithm using dynamic queues. In *Proceedings of the 2020 6th International Conference on Computing and Data Engineering*. ACM, 2020.

Epema, D. H. J. An analysis of decay-usage scheduling in multiprocessors. *ACM SIGMETRICS Performance Evaluation Review*, 23(1):74–85, 1995.

Koufaty, D., Reddy, D., and Hahn, S. Bias scheduling in heterogeneous multi-core architectures. In *Proceedings of the 5th European Conference on Computer Systems (EuroSys)*, pp. 125–138. ACM, 2010.

Linux Kernel Developers. Energy aware scheduling. Linux Kernel Documentation, 2023. https://www.kernel.org/doc/html/latest/scheduler/sched-energy.html.

Liu, C. L. and Layland, J. W. Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61, 1973.

Pabla, C. S. Completely fair scheduler. *Linux Journal*, 2009 (184):4, 2009.

Petrucci, V., Loques, O., and Mossé, D. Lucky scheduling for energy-efficient heterogeneous multi-core systems. In *Proceedings of the 2012 USENIX Workshop on Power-Aware Computing and Systems (HotPower)*. USENIX Association, 2012.

Rukmabhatla, N., Chabukswar, R., Gohad, S., and Chynoweth, M. Intel® Performance Hybrid Architecture & Software Optimizations, Part One: Introduction to Performance Hybrid Architecture for 12th Generation Intel Core Processors. Intel Corporation White Paper, Item No. 349168, 2021.

Sliwko, L. and Getov, V. Workload schedulers–genesis, algorithms and differences. *arXiv preprint arXiv:2511.10258*, 2025.

Van Craeynest, K., Jaleel, A., Eeckhout, L., Narvaez, P., and Emer, J. Scheduling heterogeneous multi-cores through performance impact estimation (PIE). In *2012 39th Annual International Symposium on Computer Architecture (ISCA)*, pp. 213–224. IEEE, 2012. doi: 10.1109/ISCA.2012.6237019.

Waldspurger, C. A. and Weihl, W. E. Lottery scheduling: Flexible proportional-share resource management. In *Proceedings of the 1st USENIX Conference on Operating Systems Design and Implementation*. USENIX Association, 1994.

Waldspurger, C. A. and Weihl, W. E. Stride scheduling: Deterministic proportional-share resource management. Technical Report MIT-LCS-TM-528, MIT Laboratory for Computer Science, 1995.